Experiment - 9

Aim : To create and deploy an ERC-20 token using Solidity in Remix IDE on a blockchain test network.

Theory :

ERC-20 is a technical standard used for creating fungible tokens on the Ethereum blockchain. Fungible tokens are interchangeable, meaning that each token of the same type has equal value and functionality.

Remix IDE is an online development environment used to write, compile, and deploy smart contracts written in Solidity. MetaMask is a cryptocurrency wallet used to connect Remix IDE with a blockchain network and approve transactions.

In this experiment, an ERC-20 token smart contract is created and deployed using Remix IDE. MetaMask is connected to a test network such as Sepolia, and test ETH is used to pay transaction gas fees. After deployment, the token's properties and functions can be tested.

Main ERC-20 Functions

1. **totalSupply():** Returns the total number of tokens in circulation.
2. **balanceOf():** Returns the token balance of a particular account.
3. **transfer():** Transfers tokens from one account to another.
4. **approve():** Allows another account to spend a specified number of tokens.
5. **allowance():** Returns the remaining amount an approved account can spend.
6. **transferFrom():** Transfers tokens on behalf of another account using an approved allowance.

ERC-20 Events

- **Transfer:** Emitted when tokens are transferred between accounts, including token minting and, in standard implementations, burning.
- **Approval:** Emitted when an account approves another account to spend tokens.

ERC-20 Attributes

- **name:** Name of the token.
- **symbol:** Short symbol representing the token.
- **decimals:** Number of decimal places used to represent token amounts.
- **totalSupply:** Total number of tokens currently in circulation.

ERC-20 and ERC-721

ERC-20 is used for fungible tokens, where each token is interchangeable with another token of the same type. ERC-721 is used for non-fungible tokens (NFTs), where each token has a unique identity.

ERC-20 Extensions

- **Mintable:** Allows new tokens to be created.
- **Burnable:** Allows tokens to be permanently destroyed.
- **Pausable:** Allows specified token operations to be temporarily stopped.

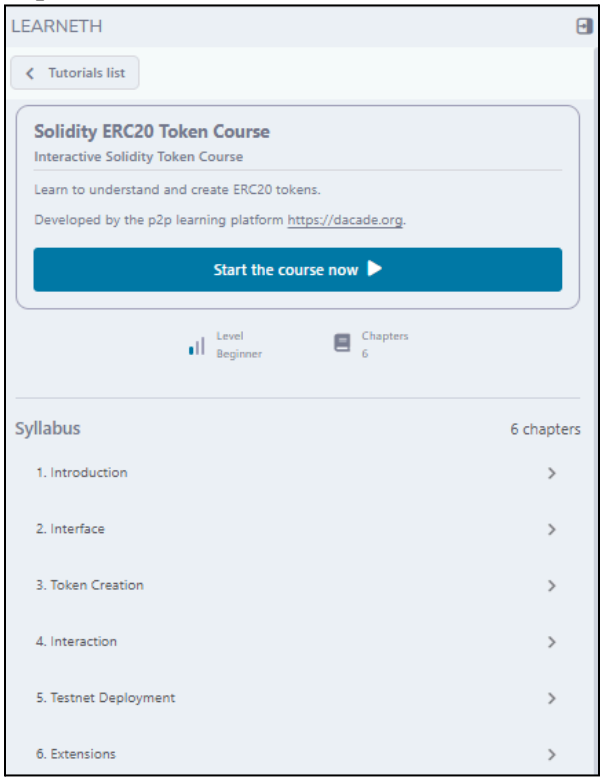
These extensions are optional and depend on the smart contract implementation.

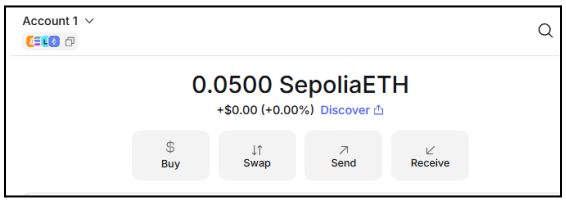
Importance of the Experiment

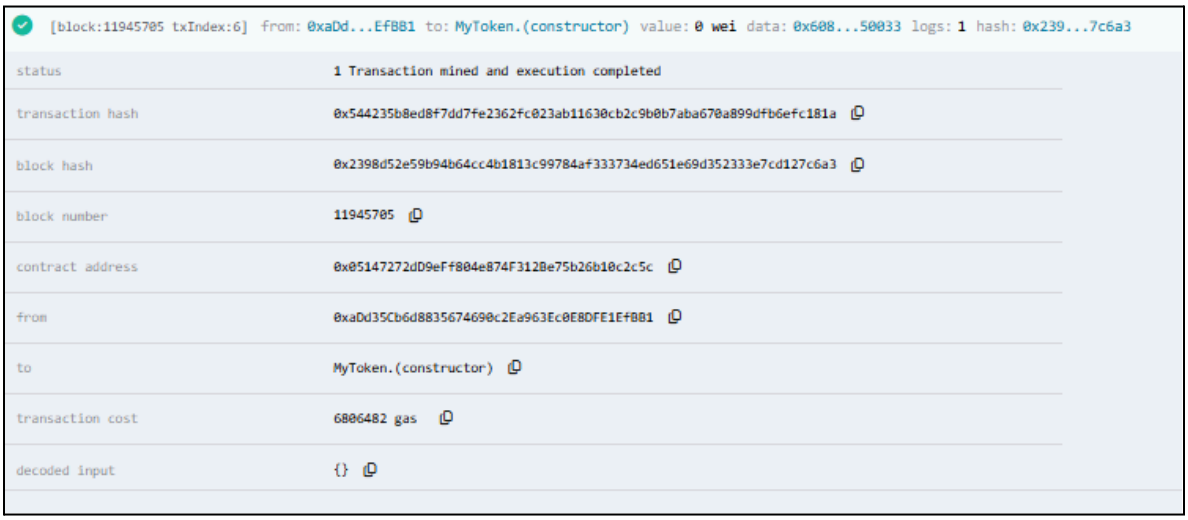
Creating an ERC-20 token helps us understand token standards, smart contract deployment, wallet integration, gas fees, and blockchain transactions. Deploying on a testnet allows us to test the token without using real cryptocurrency funds.

Implementation :

Solidity ERC20 Token Course







MyToken.sol

Introduction

Compiled | Home | introduction.sol

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract Quiz {
5      uint8 public question1 = 0;
6      uint8 public question2 = 0;
7      uint8 public question3 = 0;
8  }
```

status	1 Transaction mined and execution completed
transaction hash	0xfe6ac115f23591964c94024ae9806ff9828db8c67781225823870ec2bb922739
block hash	0xfb4f045612dcd7570c81bd4211ba1b3a4f201a3a72c167fea0d62aadb79b34cb
block number	11945576
contract address	0xA868B5b1AdfBF4a44a3dF5312FC59F1555d4Ed8B
from	0xaDd35Cb6d8835674690c2Ea963Ec0E8DFE1EfB81
to	Quiz.(constructor)
transaction cost	680632 gas

Quiz 0xA86...4Ed8B — Sepolia 21s ago

Functions

- question1 — Call
- question2 — Call
- question3 — Call

CALL [call] from: 0xadd35cb6d8835674690c2ea963ec0e8dfe1efbb1 to: Quiz.question1() data: 0x10c...d83c0

from	0xaDd35Cb6d8835674690c2Ea963Ec0E8DFE1EfB81
to	Quiz.question1() 0xA868B5b1AdfBF4a44a3dF5312FC59F1555d4Ed8B
input	0x10c...d83c0
output	0000000000000000000000000000000000
decoded input	{}
decoded output	{ "0": "uint8: 0" }
logs	[]
raw logs	[]

Interface

Compiled | Home | erc20interface.sol | introduction.sol

```
1  // SPDX-License-Identifier: MIT
2  // OpenZeppelin Contracts v4.4.0 (token/ERC20/IERC20.sol)
3
4  pragma solidity ^0.8.0;
5
6  /**
7   * @dev Interface of the ERC20 standard as defined in the EIP.
8   */
9  interface IERC20 {
10     /**
11      * @dev Returns the amount of tokens in existence.
12      */
13     function totalSupply() external view returns (uint256);   - gas
14
15     /**
16      * @dev Returns the amount of tokens owned by `account`.
17      */
18     function balanceOf(address account) external view returns (uint256);   - gas
19
```

transferFrom (address, address, uint256)

- sender (address)
- recipient (address)
- amount (uint256)

Calldata | Params

Auto-Fill with AI Samples — Transact

allowance (address, address) — Call

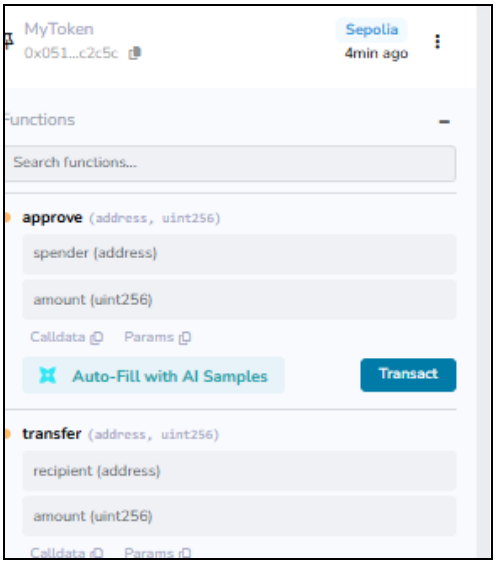
- owner (address)
- spender (address)

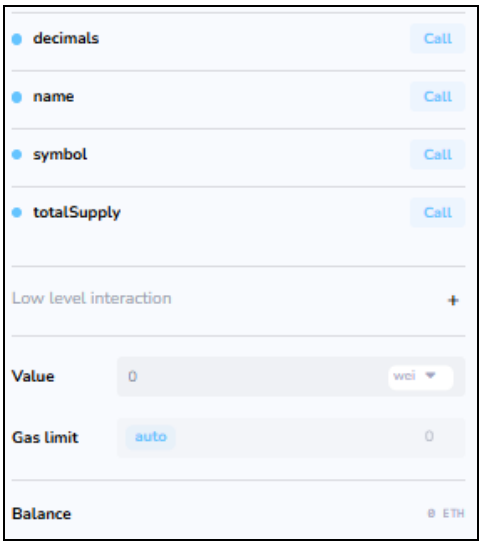
Calldata | Params

Auto-Fill with AI Samples

balanceOf (address) — Call

- account (address)

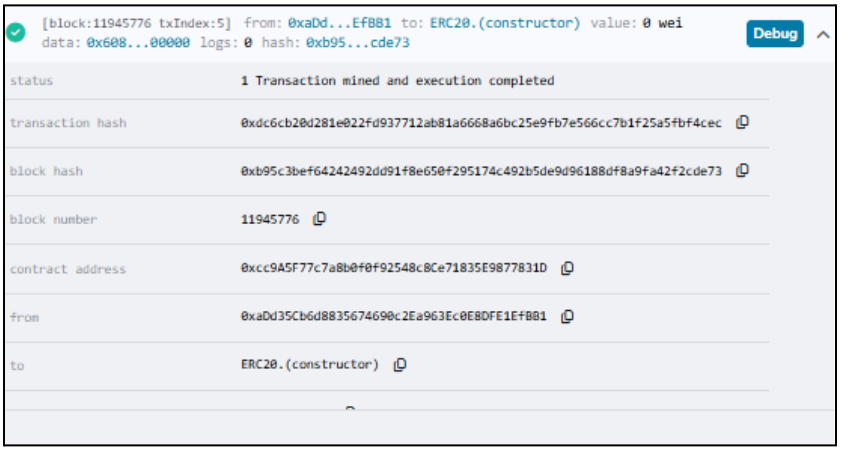


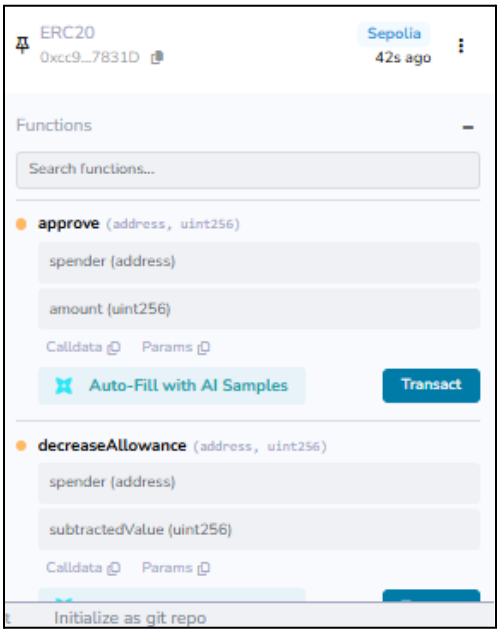


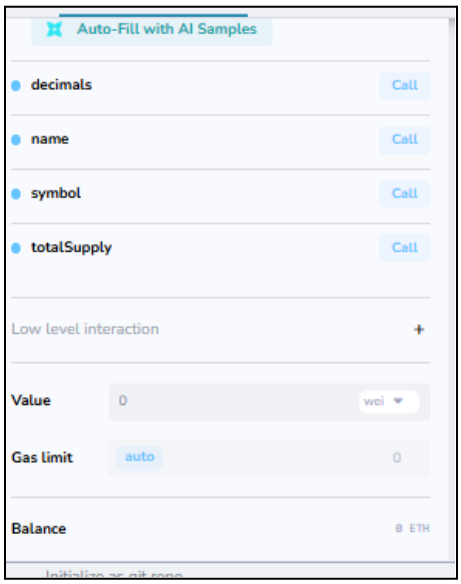


Token Creation





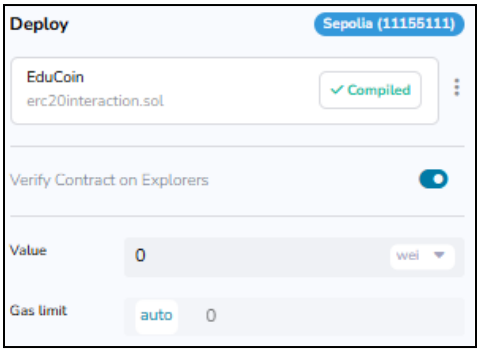






Interaction





Testnet Deployment

Testnet deployment is the process of deploying a smart contract on a test blockchain network instead of the main blockchain network. It allows developers to test smart contracts without using real cryptocurrency.

In this experiment, the ERC-20 token smart contract is compiled using Remix IDE and deployed on the Sepolia testnet. MetaMask is connected to Remix IDE to approve the deployment transaction. Test ETH is used to pay the gas fee. After successful deployment, the contract address is generated, and the token functions can be tested.

Advantages of Testnet Deployment:

- It allows safe testing of smart contracts.
- It helps identify errors before mainnet deployment.
- It uses test cryptocurrency instead of real funds.
- It helps verify token functions and transactions.

Extensions

Extensions are additional features that can be added to a smart contract to improve its functionality. In ERC-20 token contracts, extensions provide extra capabilities beyond the basic token functions.

Common ERC-20 extensions include:

1. Mintable: Allows authorized users to create new tokens and increase the total token supply.
2. Burnable: Allows tokens to be permanently destroyed, reducing the token supply.
3. Pausable: Allows authorized users to temporarily stop certain token operations.
4. Permit: Allows token approvals through a cryptographic signature instead of requiring a separate on-chain approval transaction.

These extensions are optional and depend on the smart contract implementation. They help developers customize token functionality according to project requirements.

Result

The ERC-20 token smart contract was successfully compiled and deployed on the Sepolia testnet using Remix IDE and MetaMask. The token properties and functions were examined, and the deployment transaction was verified.

Conclusion

Thus, the experiment helped us understand ERC-20 token creation, testnet deployment, and the use of MetaMask for blockchain transactions. We also learned about ERC-20 extensions and their additional functionalities.